MIT Art Design and Technology University

MIT School of Computing, Pune

Cloud Storage

Class - L.Y. (SEM-I), BDCE

Unit - IV
DISTRIBUTED STORAGE

Prof. Dr. Sonali Deshpande

AY 2026-2027 SEM-I

Unit IV - Syllabus

MIT-ADT UNIVERSITY
PUNE, INDIA

Unit IV – DISTRIBUTED STORAGE **09 hours**

Overview of Cloud Storage, structural model of the cloud storage system, Storage layer, Basic management layer, Application interface layer, Access layer, Hadoop Distributed File System (HDFS), Peer Storage System, Comparison of traditional client/server model and the peer-to-peer storage system. Managing Data Infrastructures for Cloud and Virtual Environments.

Overview of Cloud Storage

- Internet giants like Amazon, Google, Alibaba, Baidu, and Tencent have propelled the development of cloud computing, big data, and artificial intelligence technologies.
- The infrastructure supporting applications provided by these companies aims to create distributed storage systems that are high-performing, cost-effective, scalable, and user-friendly.
- While distributed storage systems have existed for years, recent advancements in big data and AI applications have led to widespread adoption of these systems in engineering practices.
- A new generation of distributed storage systems prioritizes low cost and scalability, aligning with the needs of the Internet industry.
- Internet companies have reshaped the landscape of large-scale distributed storage systems, driving innovation and efficiency in cloud storage solutions.

Overview of Cloud Storage

- Cloud storage is an extension of cloud computing, emphasizing the "cloud" aspect while offering distinct functionality from the computing power of cloud computing.
- It integrates various storage devices within a network using application software like cluster applications, grid technology, or distributed file systems to provide data storage and access functions.
- Unlike traditional storage devices, cloud storage comprises hardware components, network devices, servers, application software, public access interfaces, access networks, and client programs.
- Each component revolves around the storage device core, collectively providing data storage and business access services through application software.
- The structural model of a cloud storage system encompasses all these components, highlighting the complex interplay between hardware and software in facilitating storage and access functions.

Structural model of the cloud storage system

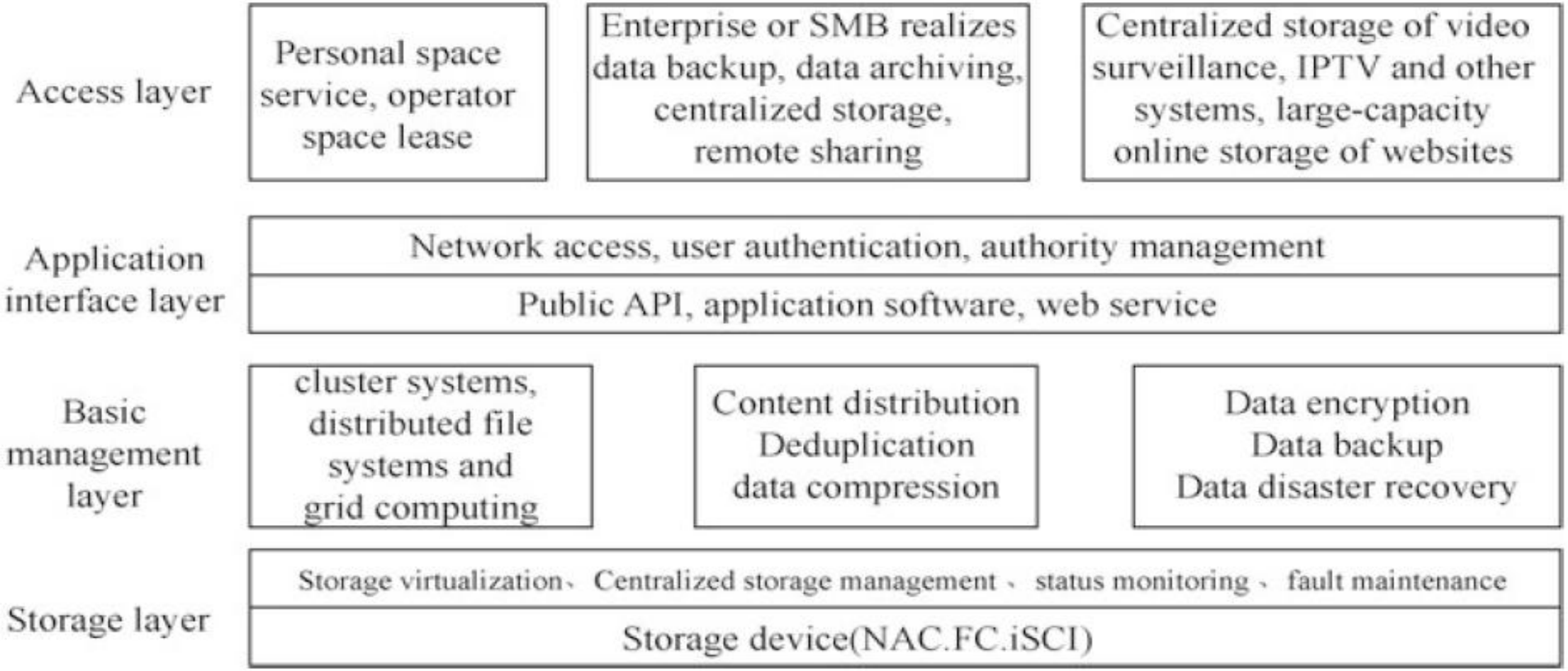


Fig. 5.37 Structure model of cloud storage system

Structural model of the cloud storage system

- The structural model of the cloud storage system consists of four layers
- **Storage Layer**: The fundamental component of cloud storage, comprising various storage devices such as FC, IP (NAS, iSCSI), or DAS (SCSI, SAS).
- These devices are often numerous and geographically distributed, interconnected via wide area networks, the Internet, or FC networks.
- A unified storage device management system sits above these storage devices, enabling logical virtualization management, multi-link redundancy management, and hardware device monitoring and fault maintenance.

Structural model of the cloud storage system

- **Basic Management Layer**: The core component of cloud storage, challenging to implement and vital for its operation.
- Utilizes cluster systems, distributed file systems, and grid computing to enable collaboration among multiple storage devices.
- Enables multiple storage devices to offer the same service, enhancing data access performance.
- Implements content distribution and data encryption technologies to prevent unauthorized access to cloud storage data.
- Incorporates various data backup and disaster recovery technologies to ensure data integrity, security, and stability within the cloud storage environment.

Structural model of the cloud storage system

- **Application Interface Layer:** The most adaptable component of cloud storage, catering to diverse needs.
- Enables development of various application service interfaces by different cloud storage operating units.
- Allows provision of different application services tailored to specific business requirements.
- Facilitates seamless interaction between users and cloud storage systems.
- Supports a wide range of applications, from basic file storage to advanced data analytics.
- Provides standardized APIs for easy integration with third-party applications.
- Offers scalability and flexibility to adapt to changing business needs.
- Ensures security and access control mechanisms to protect sensitive data.

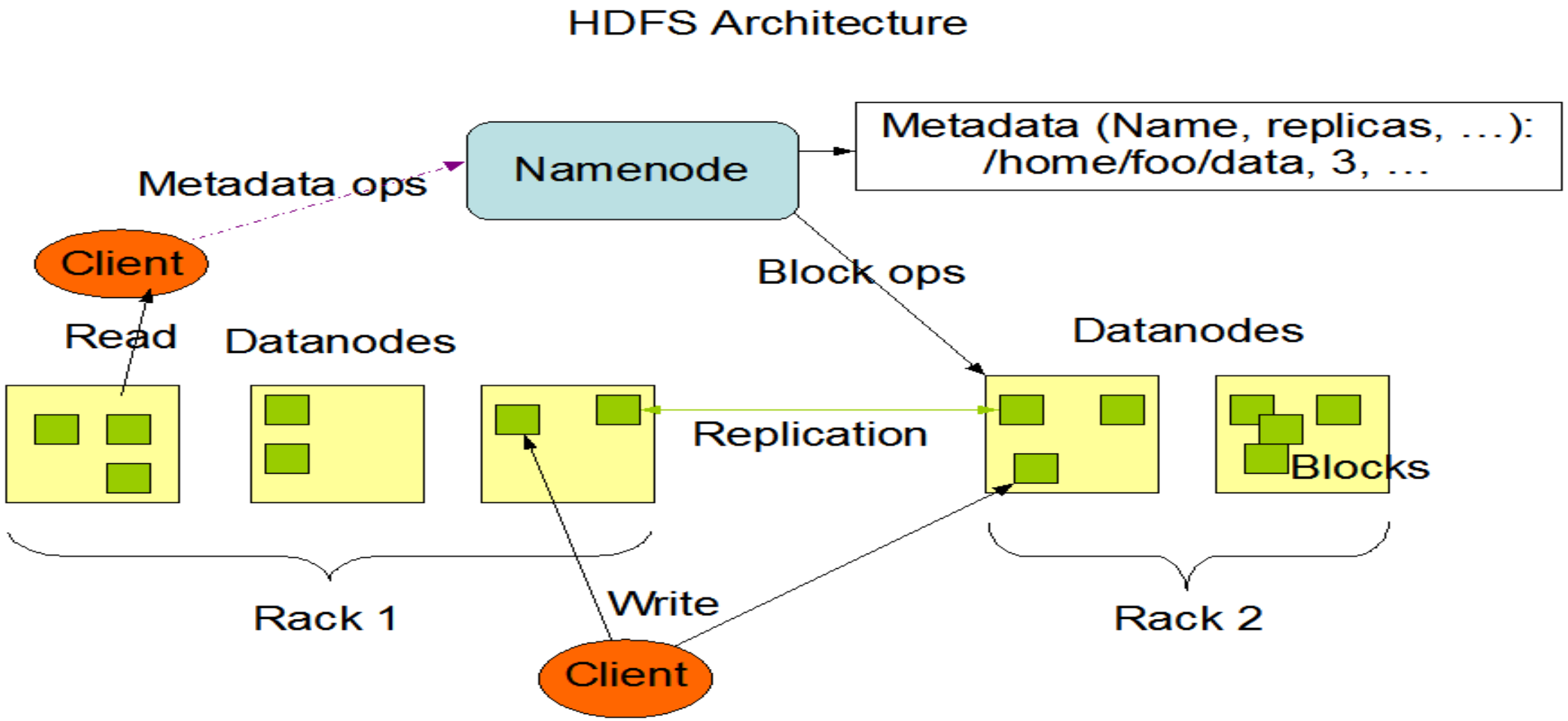
Structural model of the cloud storage system

- **Access layer:** Provides various access methods and interfaces for authorized users to interact with the cloud storage system.
- Offers flexibility in access types tailored to specific user requirements or application scenarios.
- Supports diverse applications such as video surveillance, interactive Internet TV (IPTV), video on demand, network-attached storage (NAS), and remote data backup.
- Enables seamless integration with different platforms and devices for easy data access and management.
- Implements robust authentication and authorization mechanisms to ensure secure access to cloud storage resources.
- Facilitates efficient data transfer and retrieval through optimized access protocols and technologies.
- Enables remote monitoring, control, and management of stored data from any location with internet connectivity.

Hadoop Distributed File System (HDFS)

- **Hadoop Distributed File System (HDFS)** is a distributed file system designed for storing and managing large volumes of data.
- It forms a part of the Apache Hadoop ecosystem and is optimized for handling big data across clusters of commodity hardware.
- HDFS operates on a master-slave architecture with a NameNode managing metadata and DataNodes storing data blocks.
- Data is divided into blocks and replicated across multiple DataNodes to ensure fault tolerance and reliability.
- HDFS is well-suited for batch processing and is widely used in various applications, including big data analytics and data warehousing.

Hadoop Distributed File System (HDFS)



Hadoop Distributed File System (HDFS)

- HDFS comprises a NameNode and multiple DataNodes, with the NameNode managing the file system namespace and client access, while DataNodes store data.
- Similar to a standalone file system, HDFS allows operations like creating directories, files, copying, and deleting files, but it divides files into blocks distributed across DataNodes.
- Blocks are replicated across multiple DataNodes for fault tolerance, ensuring data durability and availability.
- The NameNode maintains metadata about file-to-block mapping, facilitating file operations and block management.
- HDFS is optimized for storing and processing large datasets across distributed computing clusters, commonly used in big data applications and analytics.

Hadoop Distributed File System (HDFS)

- DataNode availability: DataNodes are typically available on common computers within the Hadoop cluster, responsible for storing and managing data blocks.
- DataNode status: The status of each DataNode, including its availability, storage capacity, and health, is obtained through regular heartbeat messages sent by DataNodes to the NameNode.
- Write process in HDFS: When writing data to HDFS, the client interacts with the NameNode to obtain metadata about the file and its block locations. The client then communicates directly with the appropriate DataNodes to write data blocks.
- Read process in HDFS: To read data from HDFS, the client contacts the NameNode to retrieve block locations. The client then fetches the required blocks directly from the DataNodes based on the obtained information.

Hadoop Distributed File System (HDFS)

- **(1) The writing process of HDFS**: NameNode Management: The NameNode in HDFS manages metadata related to all files stored within the system. It confirms client requests, records file names, and identifies the DataNodes storing each file, storing this information in a file allocation table in memory.
- Example Execution: When a client requests to write a .log file to HDFS, the NameNode oversees the process, ensuring proper allocation of file storage across DataNodes.
- Ensuring Data Consistency: A significant challenge in distributed file system design is maintaining data consistency. In HDFS, data consistency is ensured during the write phase, where the data is not considered written until the DataNode confirms that it has a copy of the file. This guarantees that clients receive consistent data regardless of which DataNode they access.

Hadoop Distributed File System (HDFS)

- **(1) The writing process of HDFS**:

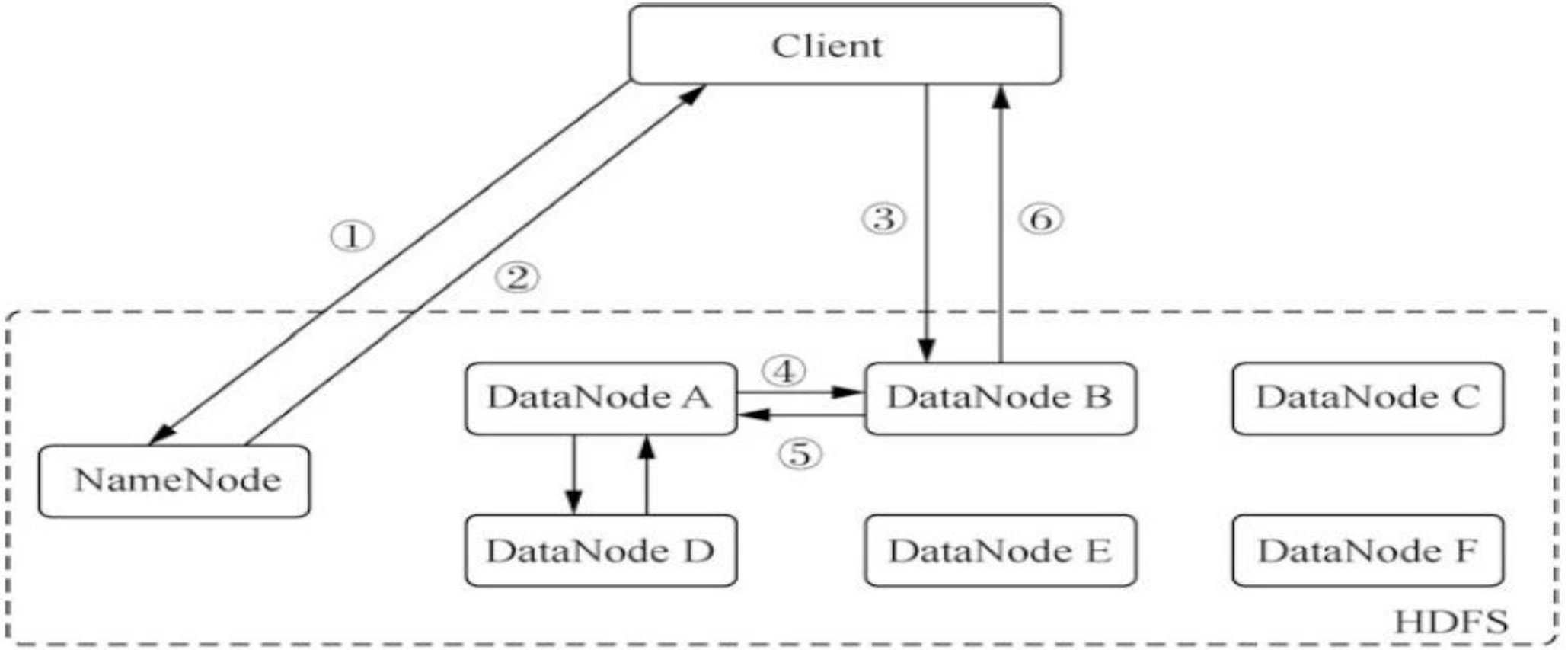


Fig. 5.39 HDFS writing process

Hadoop Distributed File System (HDFS)

- **(2) The reading process of HDFS**

- Step 1: The client sends a message to NameNode, ready to write the .log state file to HDFS (see Fig. (1))

- Step 2: NameNode sends a message to the client, commanding the client to write data nodes A, DataNode B and DataNode D and contact DataNode B directly (see Fig. (2))

- Step 3: The client sends a message to DataNode B, orders that a .log file be saved, and sends a copy to DataNode A and DataNode D (see Fig. (3))

- Step 4: DataNode A sends a message to DataNode B asking for a .log file (see Fig. (4))

- Step 5: DataNode B sends a message to DataNode A, transmits a .log file, and sends a copy to DataNode D (see Fig. (5))

- Step 6: DataNode D sends a confirmation message to DataNode A

- Step 7: DataNode A sends a confirmation message to DataNode B

- Step 8: DataNode B sends a confirmation message to the client indicating that the write is complete(see Fig. (6) in the figure)

Hadoop Distributed File System (HDFS)

- **(2) The reading process of HDFS**

- To understand the read process of HDFS, it can be thought that a file is made up of blocks of data stored on DataNode. The execution process for the client to view what was previously written is shown in Fig. 5.40. The client takes a block of files from different DataNodes in parallel, and then merges them into complete files.

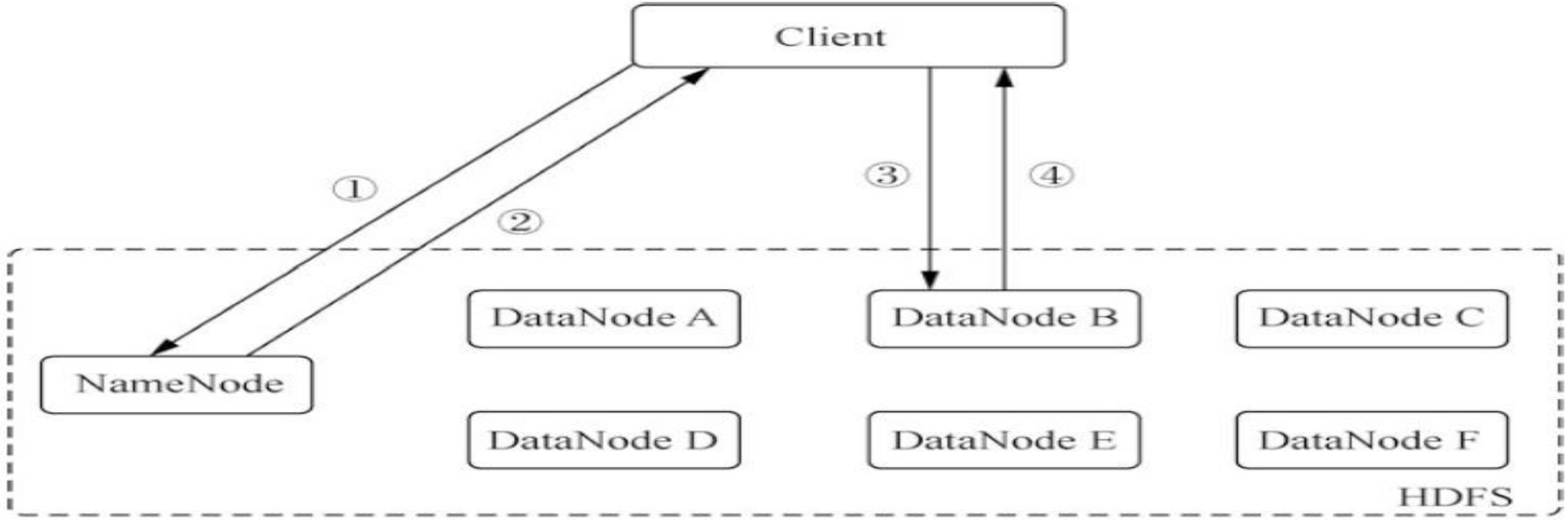


Fig. 5.40 HDFS read process

Hadoop Distributed File System (HDFS)

- **(2) The reading process of HDFS**

- Step 1: The client asks NameNode to read the file (see Fig. (1))

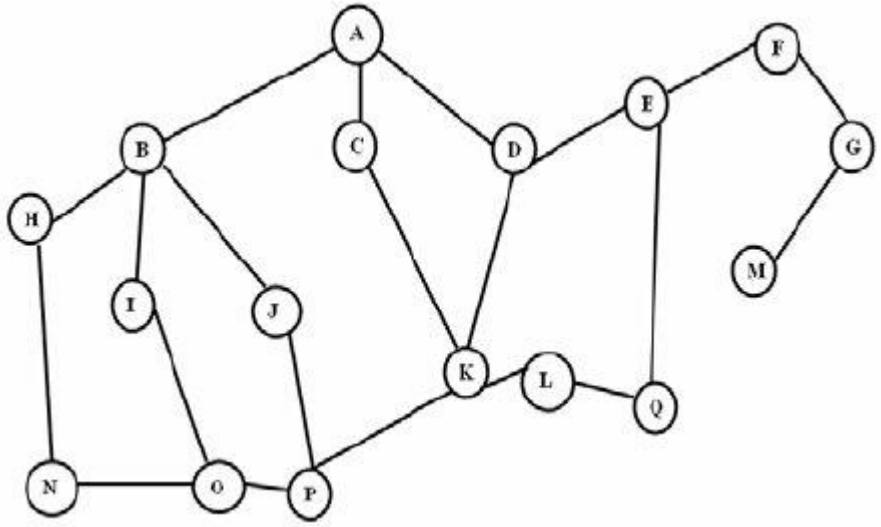
- Step 2: NameNode sends block information to the client (the block information contains the IP address of DataNode, which holds a copy of the file, and the block ID that DataNode needs to find the block on the local hard drive) (see Fig. (2))

- Step 3: The client checks the block information, contacts the relevant DataNode, and requests the block (see Fig. (3) in the figure)

- Step 4: DataNode returns the file contents to the client, then closes the connection and completes the read operation

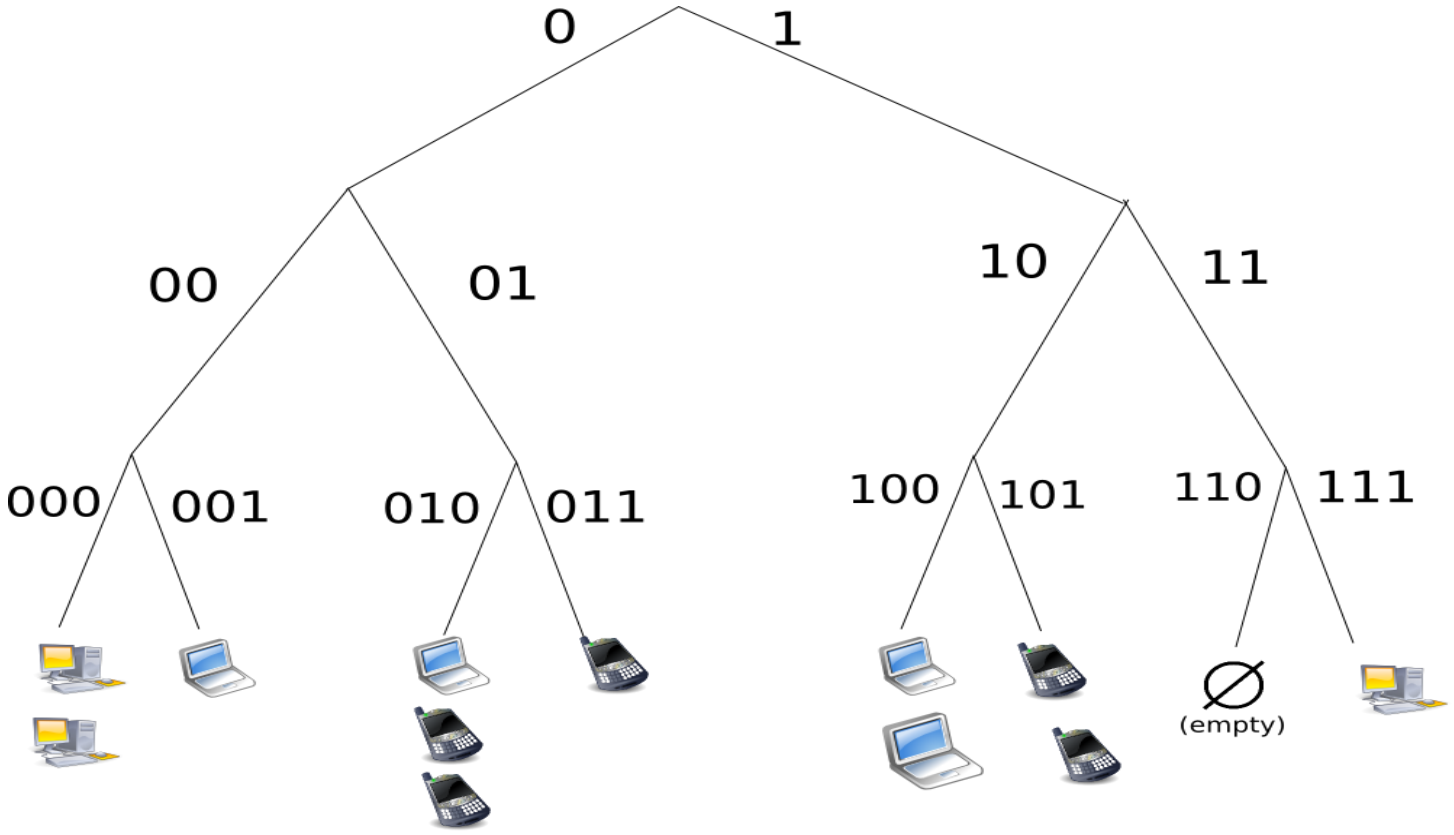
Peer Storage System

- Peer-to-peer (P2P) storage technology functions on a decentralized network, where individual computers or devices act as both clients and servers. Data is divided into fragments and stored across these devices. Here are two main types of P2P networks:
- Unstructured P2 Peers connect directly with each other to search for and share data. There's no central directory to manage file location.



Peer Storage System

- Structured P2P Leverages a Distributed Hash Table (DHT) to locate data. Each file is assigned a unique hash, and the DHT helps identify the nodes storing that specific data.



Peer Storage System

- Reciprocal Storage Network: Peer-to-peer (P2P) storage technology establishes a storage network through reciprocal interactions between storage nodes, functioning as a distributed storage system.
- Equal Status Nodes: Unlike traditional centralized control models, peer-to-peer storage systems feature storage nodes that are equal in status. This means that each node within the system has equivalent authority and capabilities.
- Composition Flexibility: A peer-to-peer storage system can consist entirely of server nodes organized in a peer-to-peer manner, desktop user nodes, or a combination of both. As long as the storage system operates in a functionally equivalent manner, it qualifies as a peer-to-peer storage system.
- Foundation in P2P Technology: Peer-to-peer storage technology is built upon the foundation of Peer-to-Peer (P2P) technology, which provides the framework and physical infrastructure for peer-to-peer storage systems to operate effectively.

Peer Storage System

- Complex Overlay Network: A peer-to-peer network is a sophisticated overlay network characterized by high dynamics. It is often referred to as an "overlay network" due to its nature of being built on top of existing physical networks.
- Virtual Layer: The peer-to-peer network serves as a virtual layer superimposed on multiple underlying physical networks. By adding this abstract layer atop the existing infrastructure, the peer-to-peer network facilitates interconnection among disparate networks while preserving their original structures.
- Resource Sharing: Through the peer-to-peer network, resources can be shared among nodes without necessitating alterations to the underlying network architecture. This enables seamless resource sharing across diverse networks.
- Distributed System: A peer-to-peer system operates as a distributed system situated at the application layer. Each node in the network communicates using a unified routing protocol, and messages traverse the network through logical connections established within the peer-to-peer network.

Comparison of traditional client/server model and the peer-to-peer storage system.

- Peer-to-peer system operates atop a peer-to-peer network, constituting a distributed system at the application layer.
- Communication between nodes occurs via a standardized routing protocol.
- Messages traverse the logical connections established within the peer-to-peer network.
- The architecture facilitates decentralized data exchange and resource sharing.
- Each node functions autonomously within the network, contributing to its resilience and scalability.
- Data transmission is facilitated through peer-to-peer network pathways, enabling efficient communication between nodes.
- The system's structure allows for dynamic and adaptable network configurations.
- Peer-to-peer networks are characterized by their peer-to-peer communication paradigm, which contrasts with traditional client-server models.

Comparison of traditional client/server model and the peer-to-peer storage system.

- - Peer-to-peer storage systems offer high scalability due to every network node functioning as both a client and a server.
- - Each node has the capability to expand its storage capacity independently, allowing for seamless scalability.
- - The system's structure enables flexible expansion and contraction of capacity without disrupting normal operations.
- - Nodes can dynamically adjust their storage resources based on demand, ensuring efficient utilization of available space.
- - Scalability is inherent in the decentralized architecture of peer-to-peer storage systems, facilitating easy adaptation to changing storage needs.
- - Unlike traditional client/server models, where scalability often requires complex server upgrades, peer-to-peer systems can scale organically with the addition of new nodes.
- - This scalability makes peer-to-peer storage systems suitable for environments with fluctuating storage requirements or rapid data growth.

Comparison of traditional client/server model and the peer-to-peer storage system.

- - Peer-to-peer storage systems offer large system capacity by pooling together free disk space from different computers.
- - Unlike traditional systems with direct mappings between data and servers, peer-to-peer systems are not limited by hardware constraints.
- - The capacity of IDE hard disks can exceed 1TB, yet the utilization rate of storage media is often less than 50%, resulting in significant wastage.
- - Peer-to-peer storage systems efficiently utilize available storage resources by pooling together unused disk space from various computers.
- - This pooled storage capacity is then shared among users who require additional storage space, maximizing resource utilization.
- - By leveraging the collective storage capacity of networked computers, peer-to-peer systems can effectively address the growing demand for storage without incurring significant hardware costs.
- - The decentralized nature of peer-to-peer storage allows for dynamic allocation of storage resources, ensuring efficient utilization and scalability.

Comparison of traditional client/server model and the peer-to-peer storage system.

- - Peer-to-peer storage systems exhibit good service performance even under unstable network conditions, thanks to data distribution across multiple nodes.
- - Changes in the number of nodes do not severely impact system performance, as each node shares the data load, adapting well to dynamic network changes.
- - The distributed nature of peer-to-peer storage allows for nearby access to data copies, reducing access latency and enhancing data retrieval capabilities.
- - Peer-to-peer storage systems boast high reliability due to their self-organizing nature, adapting to node joins and exits seamlessly while implementing file fault tolerance mechanisms.
- - Even in the event of node failures, peer-to-peer systems ensure file availability by distributing copies across the network, offering robust disaster tolerance against local catastrophic events.
- - Peer-to-peer storage systems incur low system costs as they eliminate the need for expensive file servers and network equipment, leveraging existing computing devices for storage provisioning.
- - These systems have minimal requirements for computing device performance, focusing instead on effective storage resource interconnection within the network.
- - By enabling nearby access to nodes, peer-to-peer systems minimize network traffic, particularly within local area networks, resulting in significant cost savings on external network traffic.

Managing Data Infrastructures for Cloud and Virtual Environments.

- - Data infrastructure management in cloud and virtual environments involves overseeing the storage, processing, and accessibility of data within virtualized and cloud-based systems.
- - It encompasses tasks such as provisioning and configuring storage resources, optimizing data storage and retrieval performance, and ensuring data security and compliance.
- - Managers need to consider factors like scalability, resource allocation, and integration with existing IT infrastructure when designing and managing data infrastructures for cloud and virtual environments.
- - Key responsibilities include selecting appropriate storage solutions, implementing data backup and disaster recovery mechanisms, and monitoring and optimizing data workflows.
- - Effective management of data infrastructures in cloud and virtual environments requires a deep understanding of virtualization technologies, cloud computing architectures, and data management best practices.
- - Strategies for managing data infrastructures may include automation, orchestration, and integration with cloud management platforms to streamline operations and enhance agility.
- - Additionally, managers must stay updated on emerging technologies and industry trends to continuously improve data infrastructure capabilities and address evolving business requirements.